# AI Order Intelligence for Social Commerce

Complete Project Document

System Design, Architecture & Technical Specification

| | |
|---|---|
| **Document Type** | Technical Specification & System Design |
| **Hackathon** | MSME Idea Hackathon 6.0 |
| **Theme** | Industry 4.0 and 5.0 |
| **Team Size** | 2 (Full-stack, split by feature) |
| **Deadline** | 14 July 2026 |
| **Version** | 1.0 — Finalized |

# Table of Contents

# 1. Product Overview

A tool that sits behind a small seller's WhatsApp, Instagram, Facebook, and Telegram accounts, turns messy chat orders into clean structured orders, and predicts which orders will actually get delivered — so sellers stop losing money on returns.

It is an AI assistant that reads the seller's chats, catches risky orders before they're shipped, and quietly improves the seller's delivery success rate — without asking the seller to change how they already sell.

# 2. The Problem

In India, millions of small sellers don't have a website. They sell through chat — a customer messages them on WhatsApp or Instagram, says "I want this in size M, COD," and the seller manually notes it down, ships it, and hopes for the best.

The problem: a huge number of these orders come back undelivered.

- Nationally, 25-40% of Cash-on-Delivery orders get returned (RTO — Return to Origin). For chat-based sellers, it's often 50%+.
- Every returned order costs the seller Rs. 150-300 in wasted shipping, packaging, and time.
- Big brands (Nykaa, boAt, Mamaearth) solve this with tools like GoKwik — AI that checks the checkout and predicts if an order is risky.
- Small sellers on WhatsApp/Instagram have no such tool. They have no checkout page, no buyer history, no fraud check. They're flying blind.

**The core problem:** Small chat-based sellers lose money on every bad order, and today, nothing helps them see it coming.

# 3. The Solution

We build a system that plugs into wherever a seller talks to their customers — WhatsApp, Instagram DMs, Facebook Messenger, Telegram — and does three things automatically:

**1. Understands the order**

Reads the chat (even messy Hinglish, voice notes, or vague addresses like "near Hanuman mandir, pink house") and turns it into a clean, structured order: product, price, address, payment mode.

**2. Checks the risk**

Before the seller ships, the system scores how likely this order is to actually get delivered, based on the buyer's area, order value, payment type, and past behavior (if known).

**3. Helps the seller act on it**

Confirms safe orders automatically, sends a quick confirmation message for risky ones, and suggests prepaid instead of COD when the risk is high.

Underneath, it connects to the seller's dashboard, and learns from what actually happens to each order — delivered, returned, or cancelled — to get smarter over time.

---

# 4. Target Users

- Instagram and Facebook page sellers (clothes, jewellery, home decor, etc.)
- WhatsApp-based resellers and small business owners
- Telegram-based sellers (resale, niche/drop-style selling)
- Any small seller who takes orders through chat instead of a website

These sellers are largely ignored by existing tools because those tools are built for big brands with real websites.

---

# 5. Why Now — Market Timing

- **Social commerce is exploding** — selling via chat/DM is growing rapidly in India, especially in Tier-2/3 cities.
- **COD dominates** — Cash-on-Delivery is still the primary payment method in India, and COD is exactly where RTO is worst.
- **ONDC government push** — The government's Open Network for Digital Commerce initiative is trying to formalize the exact segment of unorganized sellers this platform serves. ONDC provides the commerce rails; sellers still need tooling on top to manage orders and reduce losses.
- **Market gap is open** — Big players (GoKwik, Shiprocket) are focused on brands with websites. They haven't gone deep into chat-native selling yet. This gap exists today but won't last forever.

---

# 6. How It Works — End to End

**Step 1: Seller connects their accounts**

Seller connects their WhatsApp, Instagram, Facebook, or Telegram account to the platform via Chatwoot (open-source multi-channel messaging layer).

**Step 2: Customer messages to order**

A buyer sends a message like "bhaiya blue wala size M chahiye, COD karna hai, address — pink house hanuman mandir ke paas gomti nagar lucknow."

**Step 3: AI extracts the order**

The system reads the message, detects language, classifies intent, and extracts product, size, quantity, payment mode, and address fragments — even from messy Hinglish.

**Step 4: Address resolution**

The Address Intelligence Agent resolves vague landmarks into structured addresses with pincodes and assigns a deliverability confidence score.

**Step 5: Risk scoring**

The risk engine scores the order based on payment mode, address quality, buyer history, pincode statistics, order value, and time signals.

**Step 6: Action based on risk tier**

- **Low risk (0-35):** Auto-confirm, pass to seller as "Ready to Ship"
- **Medium risk (36-65):** Send buyer a confirmation nudge, wait for response
- **High risk (66-100):** Notify seller with warning, suggest asking for prepaid

**Step 7: Seller ships via their own courier**

The seller sees a clean order card on their dashboard and ships through whatever courier they normally use. The platform does not handle logistics.

**Step 8: Outcome feedback**

Seller marks the order as delivered, returned, or cancelled. This outcome feeds back into the risk engine, the buyer trust network, and the dreaming pipeline — making the system smarter with every order.

---

# 7. Finalized Technical Decisions

| Decision | Choice |
|---|---|
| Backend | Python FastAPI |
| AI Orchestration | LangGraph + Google ADK |
| LLMs | Multiple models per pipeline (TBD per pipeline) |
| Channel Layer | Chatwoot (open-source, self-hosted) |
| Database 1 | PostgreSQL + pgvector (orders, transactions, embeddings) |
| Database 2 | MongoDB (raw chat logs, agent traces, dream data) |
| Cache + Queue | Redis (message queue, caching, rate limiting) |
| Frontend | React.js (seller dashboard) |
| Logistics | Seller handles their own — platform presents clean orders |
| Payments | Suggest only — seller collects themselves |
| Training Data | Agent Dreaming pipeline for cold start |
| Team | 2 people, both full-stack, split by feature |
| Deployment | TBD (Railway or similar) |
| Goal | Real product — hackathon first, then continue building |

---

# 8. High-Level Architecture

```
SELLER CHANNELS (WhatsApp | Instagram | Facebook | Telegram)
|
v
CHATWOOT (Self-Hosted)
Handles all platform connections, message routing,
webhook normalization. System never talks to
Meta/Telegram APIs directly.
| Unified webhook (normalized message)
v
FASTAPI GATEWAY SERVICE
Receives Chatwoot webhooks | Authenticates
Validates | Routes to correct pipeline | Rate limiting
|
v
AI CORE (LangGraph + Google ADK)
Pipeline 1: Order Extraction Agent
Pipeline 2: Address Intelligence Agent
Pipeline 3: Risk Scoring Engine
Pipeline 4: Action Decision Agent
Pipeline 5: Buyer Trust Network
Pipeline 6: Agent Dreaming (background)
| | |
v v v
PostgreSQL MongoDB Redis
+ pgvector (Queue + Cache)
|
v
REACT.JS SELLER DASHBOARD
Unified inbox | Order cards | Risk scores | Analytics
```

# 9. The Six Core Pipelines

## Pipeline 1 — Order Extraction Agent

**Job: Read a raw chat message and extract a structured order.**

The agent detects language (Hindi, Hinglish, English, regional), classifies intent (is this an order, question, complaint, or follow-up?), and if it's an order, extracts: product name/description, variant (size, color), quantity, price (if mentioned), payment mode (COD/prepaid), and delivery address fragments. If information is missing, it flags which fields are incomplete.

**Input example:**

```
{
"seller_id": "seller_abc",
"channel": "whatsapp",
"buyer_ref": "+91-9876543210",
"raw_text": "bhaiya ye blue wala size M chahiye,\n COD karna hai, address: near hanuman\n
mandir pink house gomti nagar lucknow",
"timestamp": "2026-07-03T10:30:00Z"
}
```

**Output example:**

```
{
"intent": "new_order",
"order": {
"product_desc": "blue wala size M",
"quantity": 1,
"price": null,
"payment_mode": "COD",
"address_raw": "near hanuman mandir pink house\n gomti nagar lucknow",
"missing_fields": ["price"]
},
"confidence": 0.87,
"language_detected": "hinglish"
}
```

**LLM needed: Fast + cheap (Gemini Flash, GPT-4o-mini, or fine-tuned small model). Runs on every message.**

**Stored in: MongoDB (raw message + extraction log), PostgreSQL (structured order record)**

---

## Pipeline 2 — Address Intelligence Agent

**Job: Take messy, incomplete Indian addresses and turn them into clean, deliverable addresses with confidence scores.**

This is one of the most technically valuable components. The agent parses address fragments (landmark, area, city, state), transliterates if needed (Hindi to English), resolves into a structured address with pincode, uses pgvector to search for similar addresses seen in past orders across all sellers, and assigns a deliverability confidence score. If the address is too vague, it generates a follow-up question for the buyer.

**Input: "near hanuman mandir pink house gomti nagar lucknow"**

**Output:**

```
{
"structured_address": {
"landmark": "Hanuman Mandir",
"area": "Gomti Nagar",
"city": "Lucknow",
"state": "Uttar Pradesh",
"pincode": "226010",
"full_resolved": "Near Hanuman Mandir, Pink House,\n Gomti Nagar, Lucknow, UP 226010"
},
"confidence": 0.72,
"pincode_serviceable": true,
"similar_addresses_found": 3
}
```


**LLM needed: Stronger model (Gemini Pro or GPT-4o). Indian address resolution is genuinely hard.**

**Stored in: PostgreSQL (structured address + embedding in pgvector), MongoDB (resolution reasoning trace)**

**Why this matters:** Address quality is the single biggest predictor of RTO after payment mode. A vague address alone can cause a return even if the buyer genuinely wants the product. This pipeline alone could be a standalone product.

---

## Pipeline 3 — Risk Scoring Engine

**Job: Score every order on how likely it is to be successfully delivered.**

Phase 1 uses a transparent rules engine (no ML). Phase 2 replaces individual rules with ML trained on real delivery outcomes.

**Phase 1 — Rules Engine:**

```
Base Score: 50/100

Payment mode:
COD +20 risk
Prepaid -15 risk

Address quality:
Confidence < 0.5 +15 risk
Confidence < 0.3 +25 risk
Missing pincode +10 risk

Order value:
> Rs.3000 COD +10 risk
> Rs.5000 COD +20 risk

Buyer history (if known):
Previous RTO +25 risk
Previous successful -20 risk
First-time buyer +5 risk

Pincode signals:
High-RTO pincode +15 risk

Time signals:
Late night order +5 risk
Festival season +5 risk
```

**Output:**

```
{
"risk_score": 72,
"risk_tier": "high",
"top_risk_factors": [
"COD order above Rs.3000",
"Address confidence low (0.42)",
"First-time buyer"
],
"recommended_action": "suggest_prepaid"
}
```

**LLM needed: None for Phase 1 (pure logic/rules). Phase 2 uses a trained classifier (scikit-learn/XGBoost), not an LLM.**

**Stored in: PostgreSQL (risk score tied to order record)**

---

## Pipeline 4 — Action Decision Agent

**Job: Based on risk score, decide what to do and execute it through Chatwoot.**

Decision logic based on risk tiers:

```
Risk Score 0-35 (LOW):
Auto-confirm order
```

```
Send buyer confirmation via Chatwoot
Create order card as "Ready to Ship"

Risk Score 36-65 (MEDIUM):
Send buyer confirmation nudge:
"Hi! Your order for [product] is ready.
Reply YES to confirm delivery to [address]."
Wait for response
If confirmed -> "Ready to Ship"
If no response in 24h -> "Unconfirmed"

Risk Score 66-100 (HIGH):
Notify seller on dashboard with warning
Suggest asking for prepaid
Seller makes final call
```

**LLM needed: Small model for generating natural-language messages in buyer's language (Gemini Flash / GPT-4o-mini).**

**Messages sent through: Chatwoot's API (not directly to platforms).**

**Stored in: PostgreSQL (action taken, buyer response, final outcome)**

---

## Pipeline 5 — Buyer Trust Network

**Job: Build a shared, anonymous trust layer across all sellers on the platform.**

Every completed order contributes anonymized signals. No seller sees another seller's orders. But the system learns patterns across the entire network.

**Signal structure per order outcome:**

```
{
"buyer_hash": "sha256(phone+salt)",
"pincode": "226010",
"order_value_bucket": "1000-3000",
"payment_mode": "COD",
"outcome": "delivered",
"address_confidence": 0.72,
"channel": "whatsapp",
"timestamp": "2026-07-15"
}
```

**What the network learns:**

- This buyer_hash has 4 successful deliveries across 3 sellers -> low risk
- This pincode has 45% RTO rate across all sellers -> high-risk area
- COD orders above Rs.5000 from this city have 60% RTO -> flag these

**Privacy safeguards:**

- Buyer identity is hashed, never stored in plaintext in the trust layer
- Signals are aggregated (no single seller's data is exposed)
- Compliant with India's DPDP Act — consent collected at onboarding
- Sellers can opt out of contributing (but lose access to network signals)

**LLM needed: None — data aggregation + statistical analysis + pgvector similarity matching.**

**Stored in: PostgreSQL + pgvector (trust scores, pincode stats, buyer embeddings)**

**This is the real moat.** Over time, the network effect makes the platform more valuable with every seller who joins. A new competitor would need thousands of sellers before they could match this signal.

---

## Pipeline 6 — Agent Dreaming (Background Process)

**Job: Generate synthetic training data when system is idle, to bootstrap the risk model and handle edge cases.**

**When it runs: Background process — nights, low-traffic periods.**

**How it works:**

```
Step 1: Dream Scenario Generation
AI agent creates realistic order scenarios:
- Random buyer profiles (city tier, order history)
- Random products (different RTO patterns by category)
- Random addresses (clean to vague to broken)
- Random conditions (festivals, late night, high value)

Step 2: Outcome Simulation
Based on published RTO statistics, rules engine
logic, and any real data collected so far,
predict: would this order deliver or return?
```

```
Step 3: Edge Case Hunting
Generate hard cases that stress-test the rules:
- Good buyer, new city
- Low-value COD, good pincode, first-time buyer
- High-value prepaid, vague address

Step 4: Store as Training Data
Tagged as "synthetic" (never mixed with real data
without explicit flag). Used to pre-train ML model
in Phase 2.

Step 5: Self-Improvement Loop
Compare dream predictions vs actual outcomes.
Identify where dreams were wrong.
Adjust dream parameters.
Dreams get more realistic over time.
```

**LLM needed: Creative/reasoning model for scenario generation. Runs in background so latency doesn't matter — good candidate for free-tier models.**

**Stored in: MongoDB (dream scenarios, traces), PostgreSQL (synthetic training records tagged as synthetic)**

**Critical rule:** Synthetic data is always tagged. The ML model knows which data is synthetic vs real and weights them differently. Synthetic data bootstraps; real data governs.

# 10. LangGraph Graph Design

The core order-processing flow is one LangGraph graph with conditional routing. Each box is a LangGraph node. State is passed between nodes as a typed dictionary.

```
START
|
v
[classify_intent] --- not an order ---> RESPOND_GENERIC
|
is an order
|
v
[extract_order] --- missing fields ---> ASK_FOLLOWUP
| |
complete (wait for reply)
|<--------------------------------------+
v
[resolve_address]
|
v
[score_risk]
|
+-- low ------> [auto_confirm] --------> CREATE_ORDER_CARD
|
+-- medium ----> [send_nudge] ---------> WAIT_CONFIRMATION
| |
| CREATE_ORDER_CARD
|
+-- high -----> [notify_seller] -------> SELLER_DECIDES
|
+----------+----------+
| |
seller approves seller declines
| |
CREATE_ORDER_CARD CANCEL_ORDER
```

The graph handles "wait for reply" cases by persisting state and resuming when the next message arrives from the same conversation thread.

# 11. Google ADK — Where It Fits

Google ADK is used for standalone, specialized agents that have their own tools and memory, separate from the main LangGraph order flow.

**Address Intelligence Agent (ADK)**

- Has tools: geocoding API, pincode lookup, pgvector similarity search
- Has its own memory: patterns learned about messy Indian addresses
- Called by the LangGraph graph at the [resolve_address] node

**Dreaming Agent (ADK)**

- Has tools: read industry RTO stats, query existing order patterns, generate scenarios

- Runs independently as a background process on a scheduler
- Writes to MongoDB (dream scenarios) and PostgreSQL (synthetic training data)

**Persuasion Agent (ADK) — Future**

- Handles multi-turn buyer conversations for order confirmation and COD-to-prepaid conversion
- Has its own persona and conversation style, personalized per seller's tone
- Currently handled by templates in Pipeline 4; full ADK version planned for later

# 12. Database Schemas

## PostgreSQL Tables

**sellers**

```
id, name, channels_connected[], plan, created_at
```

**orders**

```
id, seller_id, buyer_hash, channel, product_desc, quantity,
price, payment_mode, address_raw, address_resolved,
address_confidence, pincode, risk_score, risk_tier,
action_taken, buyer_confirmed, outcome (delivered/rto/cancelled),
is_synthetic (boolean), created_at, updated_at
```

**buyer_trust**

```
buyer_hash, total_orders, successful_deliveries, rto_count,
avg_order_value, last_seen, trust_score, updated_at
```

**pincode_stats**

```
pincode, city, state, total_orders, rto_rate,
avg_delivery_time, serviceability, updated_at
```

**address_embeddings (pgvector)**

```
id, address_text, embedding (vector), pincode,
times_seen, avg_delivery_success, seller_id
```

## MongoDB Collections

**raw_messages**

```
chatwoot_message_id, seller_id, channel, raw_payload,
extracted_intent, extraction_result, llm_model_used,
tokens_used, latency_ms, timestamp
```

**agent_traces**

```
pipeline_name, order_id, input, output, reasoning_steps,
llm_model, tokens, latency, errors, timestamp
```

**dream_sessions**

```
session_id, scenarios_generated, outcomes_predicted,
edge_cases_found, model_used, duration, timestamp
```

**dream_scenarios**

```
session_id, scenario (full synthetic order),
predicted_outcome, actual_outcome (null until validated),
confidence, tags[], timestamp
```

**llm_call_logs**

```
pipeline, model, prompt_tokens, completion_tokens,
cost_estimate, latency_ms, success, error, timestamp
```

# 13. Redis — Why It's Needed

Redis serves three critical roles in the architecture:

**1. Message Queue**

Chatwoot webhook fires, Gateway puts the message on a Redis queue, LangGraph workers pick it up. This decouples ingestion from processing. If AI pipelines are slow (LLM calls take 2-5 seconds), webhooks don't timeout.

**2. Caching**

Pincode stats, buyer trust scores, and risk rules get read on every single order. Hitting PostgreSQL every time is wasteful. Cache hot data in Redis with appropriate TTLs.

**3. Rate Limiting**

Multiple LLM APIs are called per order. Redis tracks call counts per minute to avoid hitting provider rate limits and manages cost budgets.

# 14. End-to-End Data Flow — Real Example

**Characters:** Priya (kurti seller in Jaipur, uses WhatsApp + Instagram). Rahul (23-year-old buyer in Lucknow).

---

**Step 1 — Rahul messages Priya on WhatsApp**

```
"Hi didi, wo blue floral kurti dikhaya tha story me,
size M hai kya? COD ho jayega?
Address: pink house, hanuman mandir ke paas,
gomti nagar, lucknow"
```

**Step 2 — Chatwoot receives and normalizes**

WhatsApp Business API delivers message to Chatwoot. Chatwoot fires a normalized webhook to FastAPI Gateway. Gateway puts it on Redis queue.

**Step 3 — Pipeline 1: Order Extraction**

LLM reads Hinglish message. Classifies as new_order. Extracts: product (blue floral kurti), size (M), payment (COD), address fragments. Flags missing field: price.

**Step 4 — Pipeline 2: Address Intelligence**

Resolves "pink house, hanuman mandir ke paas, gomti nagar, lucknow" into structured address with pincode 226010. Confidence: 0.68 (decent but no street name). Checks pgvector for similar past addresses.

**Step 5 — Pipeline 3: Risk Scoring**

```
Starting: 50
COD: +20 | First-time buyer: +5 | Address 0.68: +8
Pincode 226010 (28% RTO): +5
Final: 88/100 — HIGH RISK
```

**Step 6 — Pipeline 4: Action Decision**

Score 88 = high risk. Dashboard shows warning to Priya. System sends Rahul: "Your order for blue floral kurti (Size M) is being processed. Reply YES to confirm delivery to Gomti Nagar, Lucknow."

**Step 7 — Rahul confirms**

Rahul replies "yes confirm karo." Pipeline 1 classifies as order_confirmation. LangGraph resumes from WAIT_CONFIRMATION state. Order status: Confirmed.

**Step 8 — Priya ships**

Priya sees order card on dashboard with risk warning + buyer confirmation. She confirms price (Rs.699), decides to ship. Ships via her own courier. Marks as "Shipped."

**Step 9 — Outcome recorded**

Courier delivers. Rahul accepts and pays. Priya marks order as "Delivered."

**Step 10 — Network learns**

Buyer Trust Network records anonymous signal. Rahul's hash now has 1 successful delivery — lower risk next time. Pincode 226010 stats update. The resolved address stores in pgvector for future matching.

**Step 11 — Dreaming agent learns**

That night, Dreaming Agent compares: predicted score 88 (high risk), actual outcome delivered. Learns that buyer confirmation is a strong positive signal. Adjusts future dream parameters.

## 15. What Makes This Different

- **No website required** — works inside the chat apps sellers already use
- **Channel-agnostic** — WhatsApp + Instagram + Facebook + Telegram, unified in one system
- **Network effect** — shared anonymous buyer trust signals that no individual seller could build alone
- **Indian-language native** — handles Hinglish, Hindi, and regional languages out of the box
- **Privacy-preserving** — hashed buyer identities, aggregated signals, DPDP compliant
- **Starts with rules, graduates to ML** — transparent and explainable on day one, gets smarter with real data
- **Agent Dreaming** — novel approach to cold-start problem using synthetic scenario generation
- **Address Intelligence** — solves India's broken address system with AI, potentially a standalone product/API

## 16. Known Challenges & Mitigation

| Challenge | Mitigation Strategy |
|---|---|
| No historical training data on day one | Start with rules engine (not ML).<br>Use Agent Dreaming for synthetic bootstrap.<br>Graduate to ML after real outcomes accumulate. |
| Easy to copy | Long-term moat is the network effect:<br>buyer trust data across thousands of sellers<br>cannot be instantly replicated. |
| Multiple platform APIs with different rules | Chatwoot abstracts all platform differences.<br>One core system, thin adapters per channel.<br>Adding a new channel = adding one connector. |
| Privacy and compliance | Hashed buyer identities, aggregated signals,<br>DPDP Act compliance, consent at onboarding,<br>data retention limits enforced. |
| Distribution / GTM | Reach sellers through logistics partners,<br>WhatsApp BSP providers, seller communities,<br>not individual marketing. |
| WhatsApp message policy restrictions | All automated messages go through<br>Chatwoot with pre-approved templates.<br>Rate limiting and opt-outs built in. |
| Cross-channel buyer identity matching | Fuzzy matching on name + address + order<br>pattern across channels via pgvector<br>similarity search. |

## 17. Future Enhancements — AI Roadmap

The following AI capabilities are planned additions that sit on top of the current architecture without changing it. Each becomes a new node in the LangGraph graph or a new ADK agent.

| Enhancement | Description | AI Technique |
|---|---|---|
| Voice Note Understanding | Process orders sent as<br>WhatsApp voice messages | Speech-to-text<br>(Whisper / Gemini Audio) |
| Image-Based Ordering | Match product photos to<br>seller's catalog | Vision model +<br>semantic matching |
| Smart Catalog Matching | Resolve vague references<br>like "wo blue wala" | Embedding similarity<br>search |
| Conversation Intelligence | Detect haggling, intent<br>strength, cancellation | Sentiment + intent<br>classification |

| | | |
|---|---|---|
| COD-to-Prepaid Persuasion | AI-generated personalized messages to convert COD | A/B tested message generation |
| Fake/Duplicate Detection | Catch fraudulent or duplicate orders | Cross-seller pattern matching |
| Smart Address Autocomplete | Multi-turn conversation to fix vague addresses | Conversational agent + geocoding |
| Delivery Time Prediction | Predict when order will arrive, reduce buyer anxiety | Regression model on pincode delivery data |
| Return Reason Intelligence | Categorize and analyze why orders are returned | Classification + network analytics |
| Reorder Prediction | Predict which buyers will reorder and when | Time-series + collaborative filtering |
| Multi-Language Translation | Real-time translation between seller and buyer | Neural machine translation |
| Fraud Ring Detection | Detect coordinated fraud across buyer groups | Graph neural networks |

# 18. Shipping Timeline

### Phase 1 — Hackathon Submission (Week 1-2)

- FastAPI gateway with one channel (Chatwoot + WhatsApp or Telegram)
- Pipeline 1: Order extraction (working demo with Hinglish)
- Pipeline 3: Risk scoring (rules engine, no ML)
- Pipeline 4: Basic action (confirm / flag on dashboard)
- React dashboard: order cards with risk scores
- Concept note + block diagram for MSME submission

### Phase 2 — MVP v1 (Month 1-2)

- All four channels via Chatwoot
- Pipeline 2: Address intelligence (full)
- Pipeline 5: Buyer trust network (basic version)
- Pipeline 6: Agent dreaming (first version)
- Seller onboarding flow
- Dashboard with analytics and multi-channel unified inbox

### Phase 3 — Product v2 (Month 3-6)

- ML model replaces rules engine (trained on real + dream data)
- Full buyer trust network with cross-seller signals
- Multi-language support beyond Hindi/English
- ADK persuasion agent for buyer conversations
- Voice note and image-based order processing
- ONDC integration exploration
- Fraud ring detection via graph analysis

# 19. Open Decisions

| Decision | Status |
|---|---|
| Specific LLM models per pipeline | TBD — depends on free tier availability and testing |
| Exact deployment platform | TBD — Railway vs alternatives |
| Chatwoot: self-host vs cloud | Need to research — self-hosted gives control, cloud is faster to start |
| BSP for WhatsApp | TBD — Gupshup, Interakt, AiSensy (if not through Chatwoot directly) |

| | |
|---|---|
| Product name | TBD |
| Domain and branding | TBD |

## 20. The Pitch

> **"An AI order-intelligence layer for India's unorganized social commerce sellers — across WhatsApp, Instagram, Facebook, and Telegram — that catches risky orders before they're shipped, unifies scattered chat orders into one dashboard, and builds a shared, privacy-safe buyer trust network that gets smarter with every seller who joins."**

*End of Document*
*Version 1.0 — July 2026*